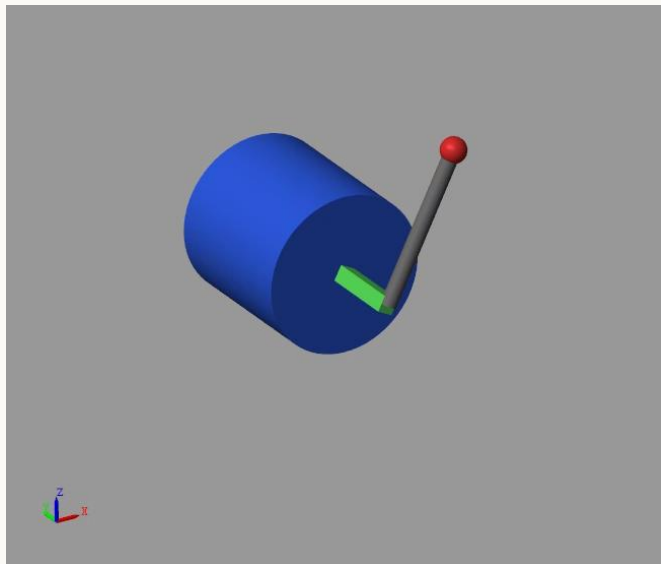


Problem Formulation

Our work focused on the speed control of a brushed DC motor with a vertical pendulum using several control techniques. To make the project accessible and easy to understand, we used Python to derive the system dynamics by modeling the mathematical equations and validated the correctness of the mathematical model using Simulink Multibody.



Motor with vertical pendulum dynamic

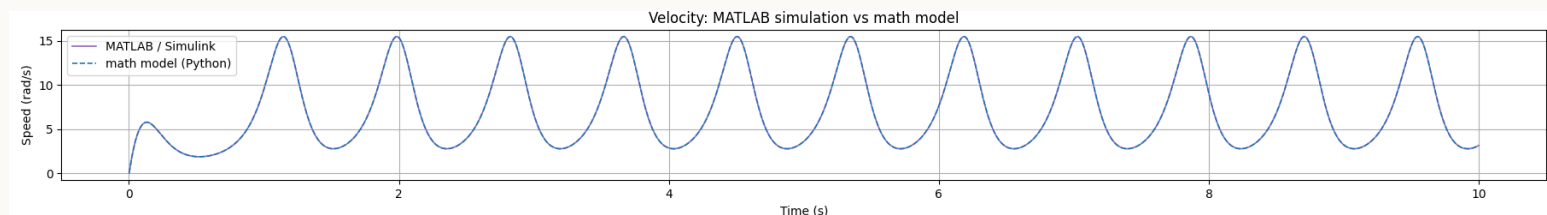
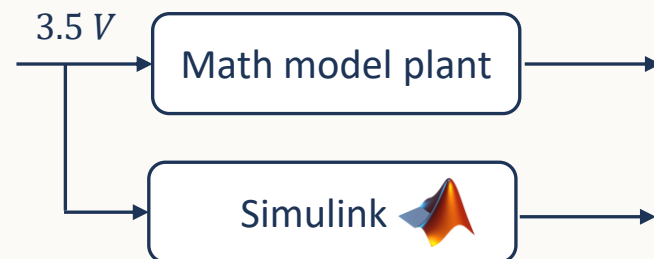
Applying Kirchhoff's voltage law to the armature circuit:

$$L \frac{di}{dt} = V_{in} - Ri - K_b \omega$$

The net torque balance on the motor shaft with an attached rod of mass

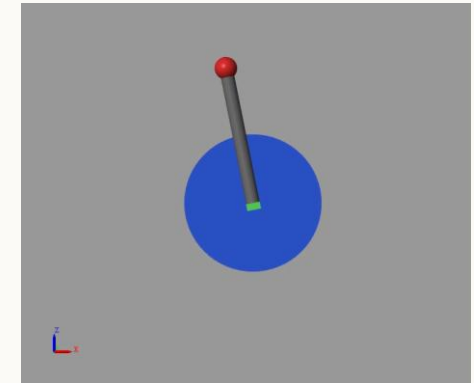
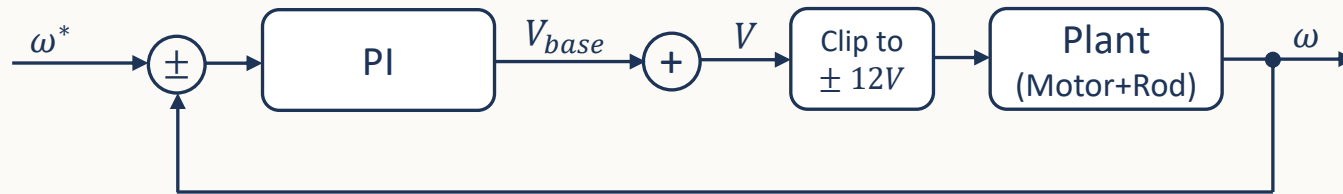
$$J_{ef} \frac{d\omega}{dt} = K_t i - B\omega - mg\ell_{cm} \sin \theta$$

Symbol	Description	Value
L	Motor inductance	4.0e-3
V_{in}	Input voltage	[-12.0, 12.0]
R	Motor Resistance	3.0
K_b, K_t	Motor Constant	0.05
J_{ef}	Total Inertia	7.04e-5
m	End effector mass	0.05
ℓ_{cm}	Rod length	0.1

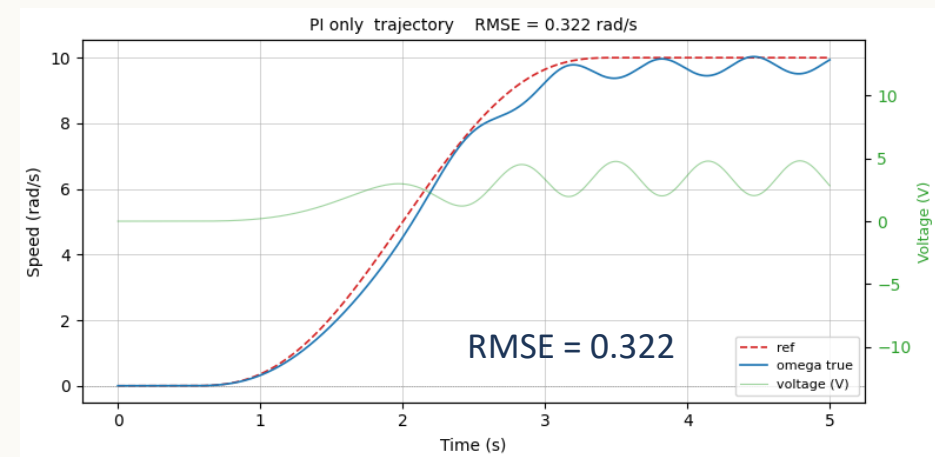
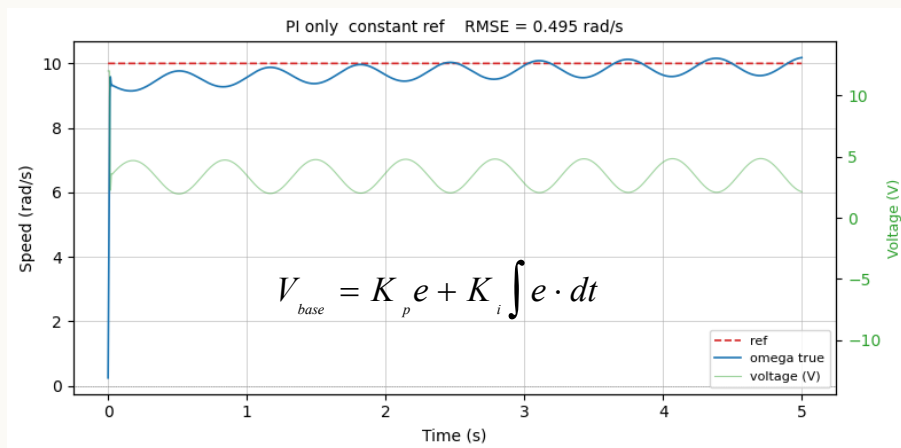


Classical Control

The classical method commonly used for speed control is the PI controller. One of its advantages is that it can provide satisfactory performance even **without a complete understanding of the plant model**. However, this approach works well only for linear systems. In our case, the presence of the vertical pendulum introduces gravitational effects, making the system highly nonlinear.

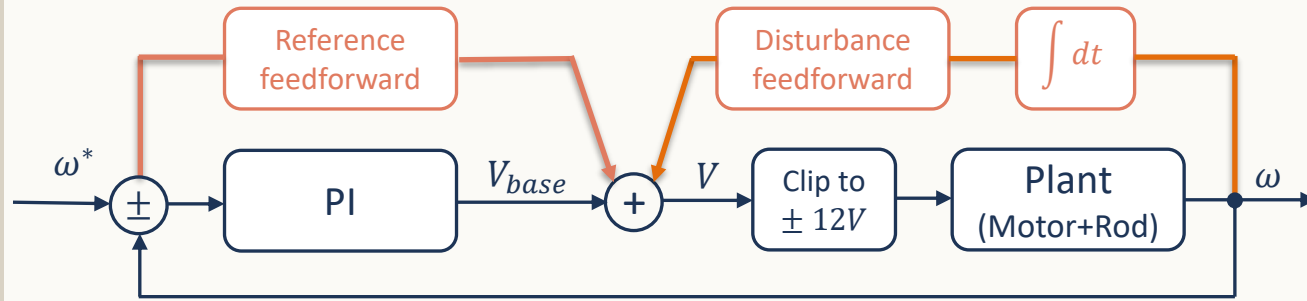


As a result, the performance of a conventional PI controller can be significantly degraded.



Classical Control

To achieve better performance, we can improve the controller **by understanding the dynamics of the system**. This allows us to incorporate feedforward control. Specifically, a disturbance feedforward term compensates for the gravitational effects, while a reference feedforward term generates the required control input to drive the system toward the desired target. Unlike a PI controller, which reacts only after an error occurs, the reference feedforward term acts proactively, enabling faster and more accurate tracking when no disturbances are present.



Feedforward Controller Design

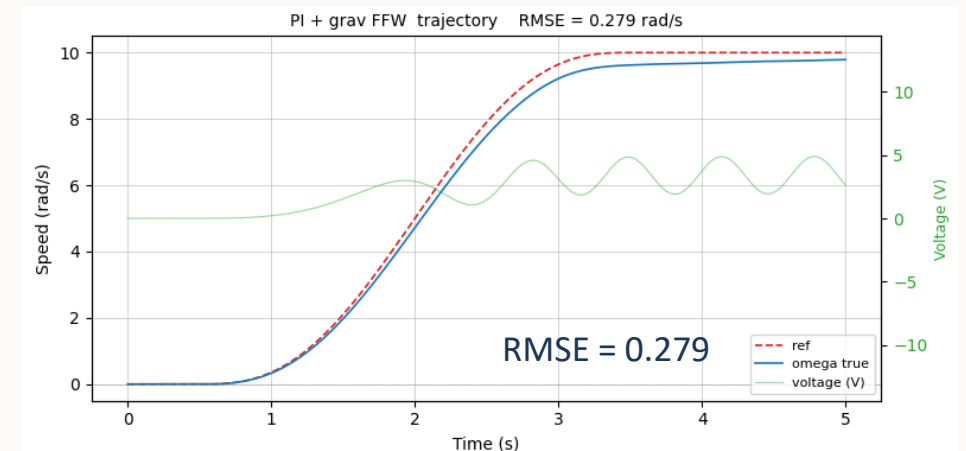
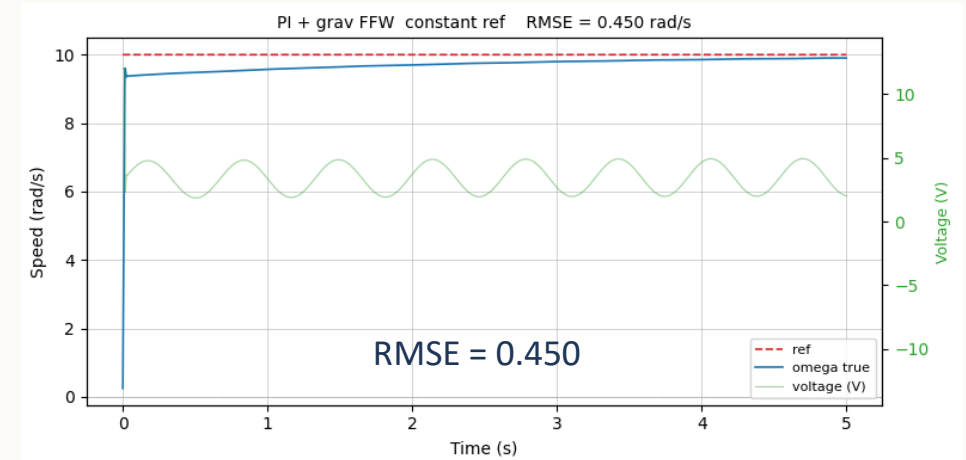
$$\text{From } L \frac{di}{dt} = V_{in} - Ri - K_b \omega \text{ and } J_{ef} \frac{d\omega}{dt} = K_t i - B\omega - mgl \sin \theta$$

Consider the steady-state condition where all derivatives vanish

$$V = Ri + K_b \omega \text{ and } 0 = K_t i - B\omega - mgl \sin \theta$$

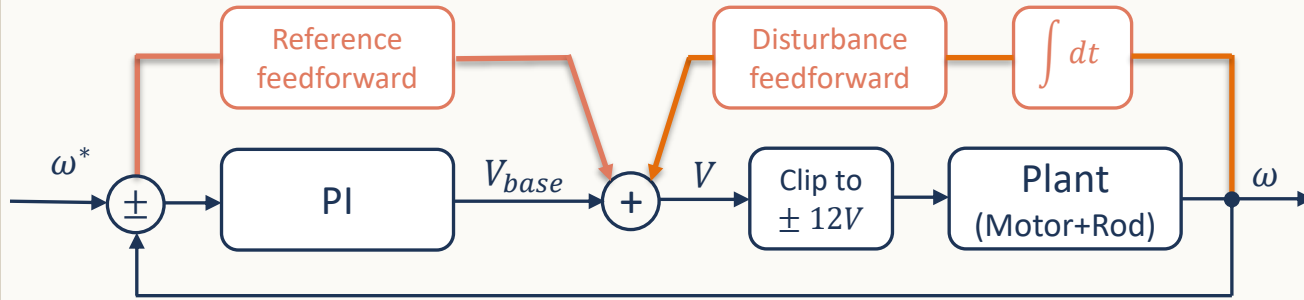
Solve this we will get the feedforward that can be written as

$$V = \underbrace{\frac{Rmgl \sin \theta}{K_t}}_{\text{Disturbance ffw}} + \underbrace{\left(\frac{RB}{K_t} + K_b \right)}_{\text{Reference ffw}} \omega$$



Classical Control

To achieve better performance, we can improve the controller **by understanding the dynamics of the system**. This allows us to incorporate feedforward control. Specifically, a disturbance feedforward term compensates for the gravitational effects, while a reference feedforward term generates the required control input to drive the system toward the desired target. Unlike a PI controller, which reacts only after an error occurs, the reference feedforward term acts proactively, enabling faster and more accurate tracking when no disturbances are present.



Feedforward Controller Design

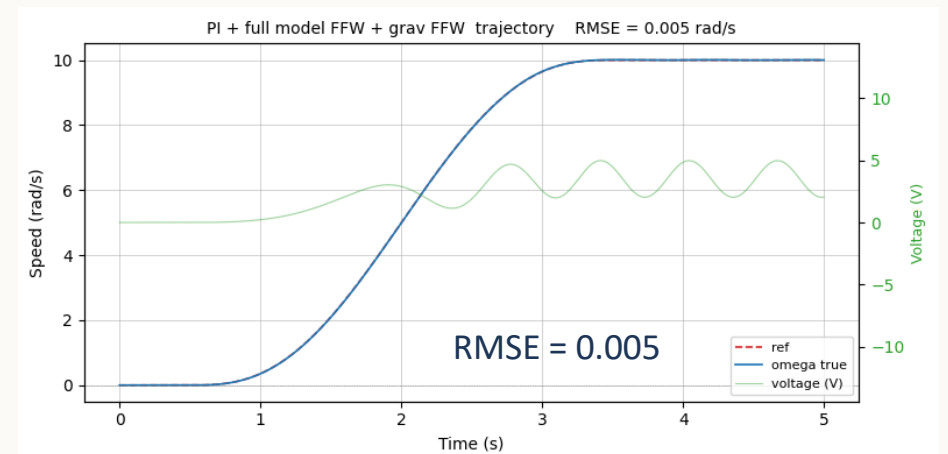
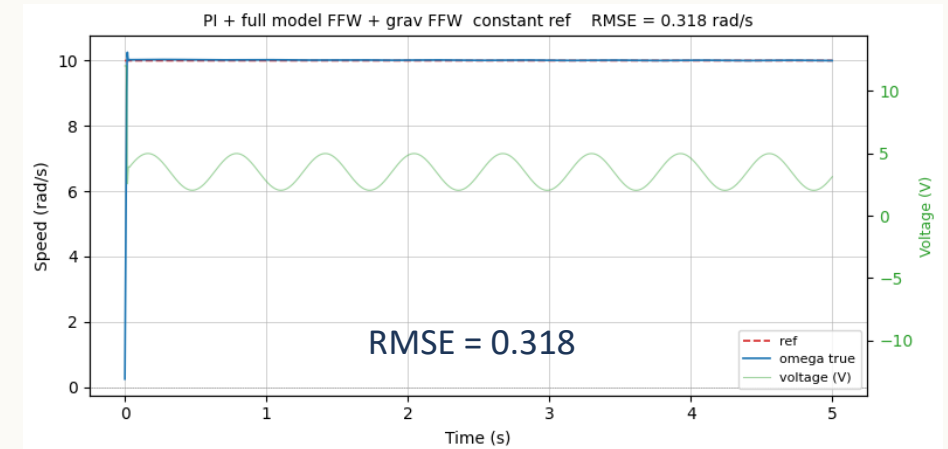
From $L \frac{di}{dt} = V_{in} - Ri - K_b \omega$ and $J_{ef} \frac{d\omega}{dt} = K_t i - B\omega - mgl \sin \theta$

Consider the steady-state condition where all derivatives vanish

$$V = Ri + K_b \omega \quad \text{and} \quad 0 = K_t i - B\omega - mgl \sin \theta$$

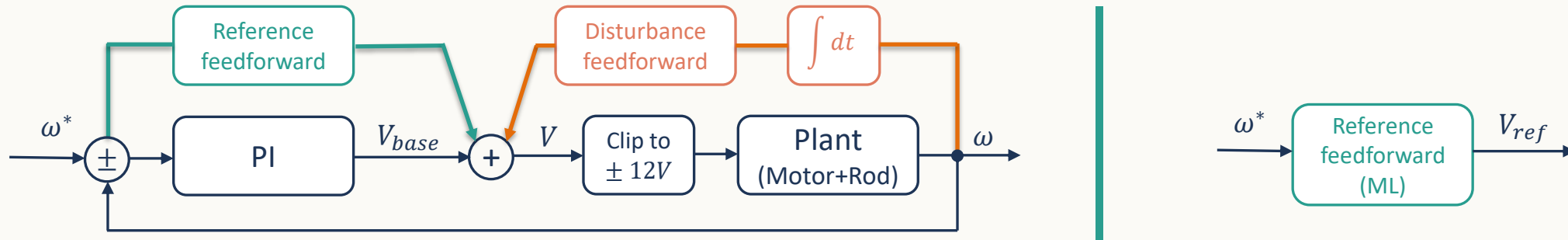
Solve this we will get the feedforward that can be written as

$$V = \underbrace{\frac{Rmgl \sin \theta}{K_t}}_{\text{Disturbance ffw}} + \underbrace{\left(\frac{RB}{K_t} + K_b \right)}_{\text{Reference ffw}} \omega$$



Machine Learning Replaced Reference Feedforward

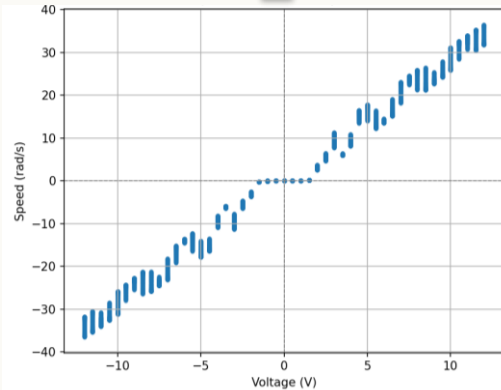
However, not every system has a well-defined or fully understood dynamic model. So, how can we still benefit from reference feedforward?



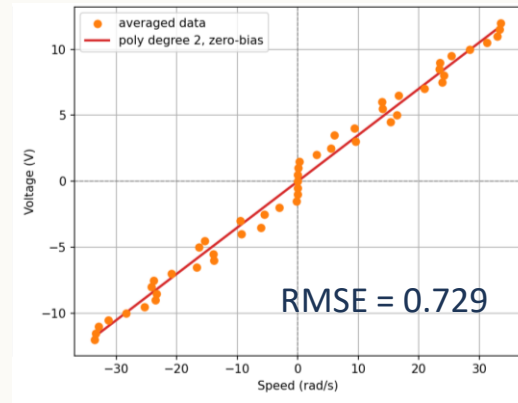
The key idea is that the reference feedforward term is essentially a function that maps the desired output, ω , to the required control input, V_{in} . **Instead of deriving this function analytically from the system dynamics, we can obtain it experimentally.** By collecting pairs of input-output data **we can use a machine learning model to learn this mapping**. Here is the step

1. Collect the motor's steady-state speed at different input voltages.
2. Average the measured speed at each voltage to reduce the effect of periodic oscillations caused by gravity.
3. Reverse the dataset to obtain the mapping from speed to voltage.
4. Train a machine learning model to learn the function ($V = f(\omega)$).
5. Use the desired speed (ω) to predict the required input voltage (V).
6. Evaluate the model's prediction accuracy and control performance.

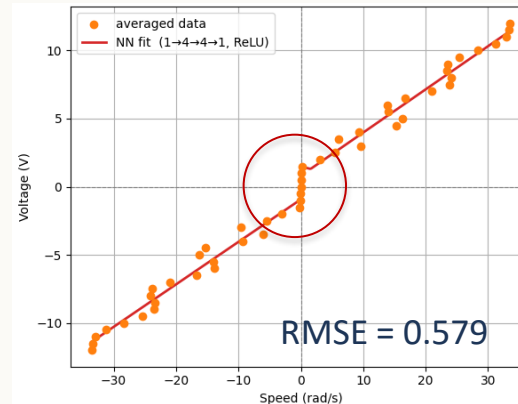
Machine Learning Replaced Reference Feedforward



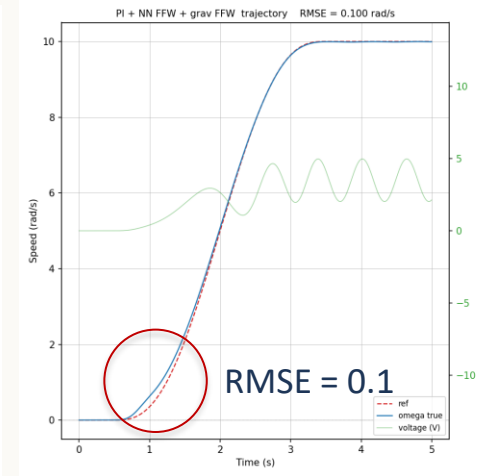
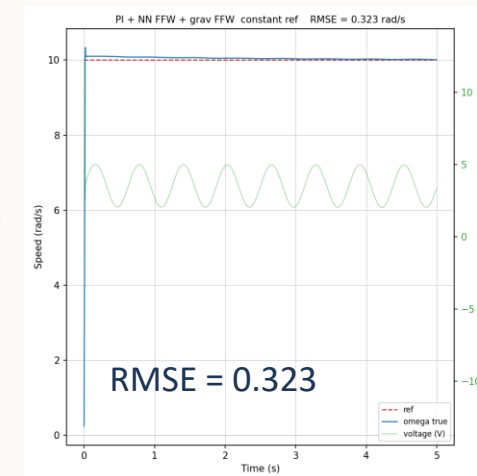
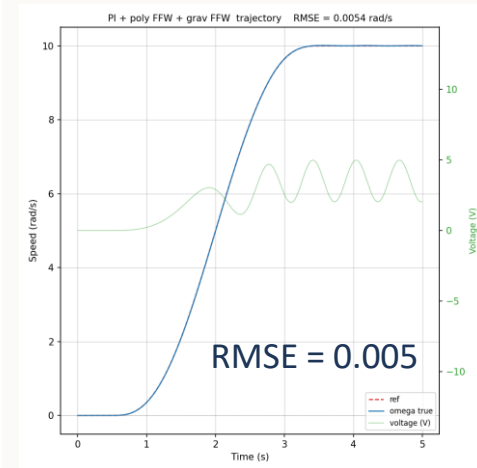
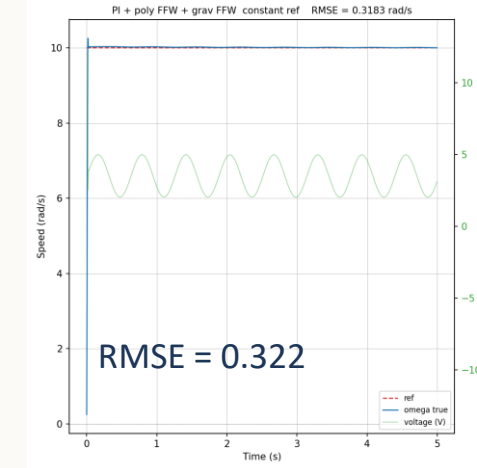
1. Collect the motor's steady-state speed at different input voltages.



Input layer: 1
Output layer: 1
Model: Linear Regression



Input layer: 1
Hidden layer: 16, 16
Output layer: 1
Epochs: 10000
Activation: ReLU
Loss: MSE



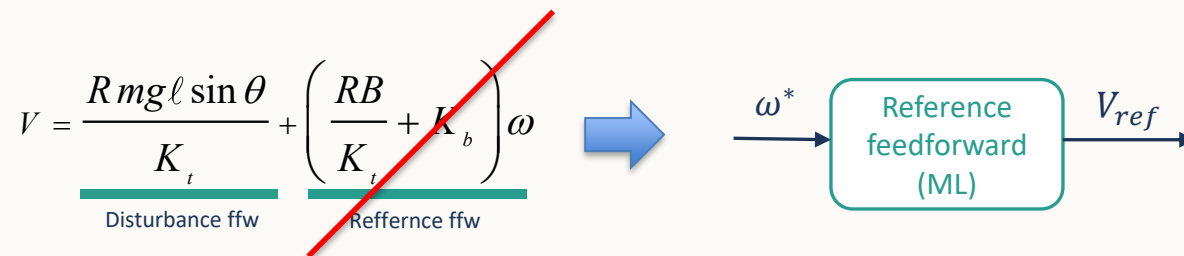
2. Average the measured speed at each voltage to reduce the effect of periodic oscillations caused by gravity.
3. Reverse the dataset to obtain the mapping from speed to voltage.
4. Train a machine learning model to learn the function ($V = f(\omega)$).



Conclusion

Metric	PI (c)	PI (t)	PI + full model FFW (c)	PI + full model FFW (t)	PI + poly FFW + grav FFW (c)	PI + poly FFW + grav FFW (t)	PI + NN FFW + grav FFW (c)	PI + NN FFW + grav FFW (t)
RMSE (rad/s)	0.4950	0.3217	0.3183	0.0053	0.3183	0.0054	0.4450	0.2714
MAE (rad/s)	0.3248	0.2476	0.0294	0.0043	0.0298	0.0044	0.2904	0.2308
Control effort	17.4124	11.1069	17.6503	11.2862	17.6509	11.2866	17.3144	11.0674

Machine learning can **reduce the need for full analytical understanding of the system**. It can achieve performance almost the same as a **fully analytical model** and already simplifies system identification by **avoiding explicit estimation of parameters such as damping (b) and back-EMF constant (k_b)**. A simple linear regression model can already reproduce **behavior close to a fully derived physical model**.



Residual Reinforcement Learning for Speed Control

A pure PI controller compensates the speed error but ignores gravity and the **nonlinear dynamics** of the system. Adding a feedforward term requires an accurate physical model. The residual approach lets us *learn* that correction directly from interaction with the system, with no knowledge of the motor parameters.

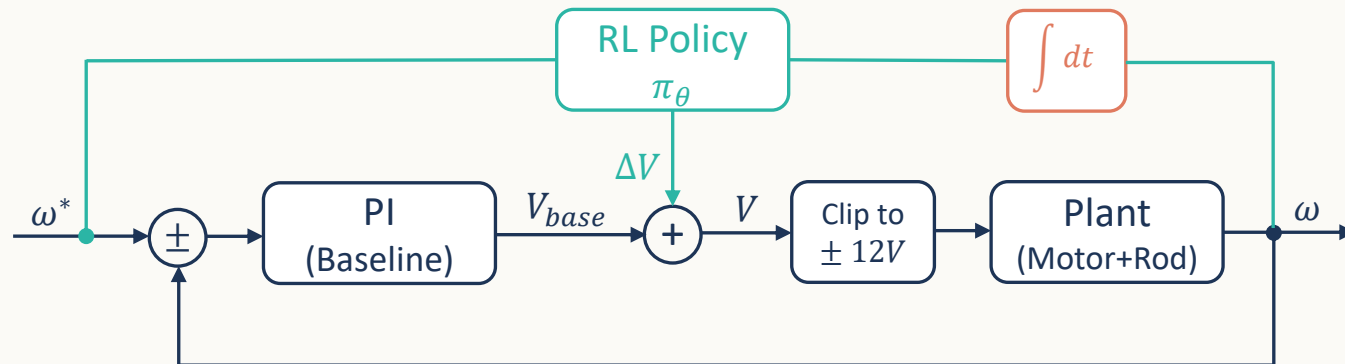


Fig. 1 - The policy does not replace the PI; it adds a bounded correction on top. Safety is guaranteed by design: $|\Delta V| \leq 3.6V$ and the total voltage never exceeds $\pm 12V$.

Agent	Neural Network (SAC) learning π_θ
Environment	BDC Motor + rod simulated at 1 kHz
State s	Speed error, velocity, $\sin/\cos \theta$, ω^* , V_{base}
Action a	$\Delta V \in [-3.6, +3.6] V$ (bounded)
Reward r	$- e $ – effort penalty – rate penalty
Objective	Accurate tracking, smooth control $\max E[\sum \gamma^t r_t]$

Main advantage. The policy learns what the PI cannot model (feedforward and gravity compensation) without ever seeing the motor equations.

State Observer

It is the interface between the system and the neural network.

At every step it collects raw measurements and converts them into the **input vector**. It does three things:

- selects only the variables needed to compute ΔV , discarding everything else;
- normalises each quantity so that training is numerically stable;
- re-encodes the rod angle as $(\sin \theta, \cos \theta)$ to avoid the discontinuity at $\pm\pi$.

$$s = \left[\frac{e}{e_s}, \frac{\omega}{\omega_s}, \sin \theta, \cos \theta, \frac{V_{base}}{V_{max}} \right] \in \mathbb{R}^6$$

e/e_s	Speed error, normalised	Primary tracking signal, tells the policy how far off we are
ω/ω_s	Current speed, normalised	Captures the current operating point and back-EMF regime
$\sin \theta, \cos \theta$	Rod angles (2 variables)	Encodes the gravity load
ω^*/ω_s	Reference speed, normalised	Lets the policy distinguish a ramp from a step, anticipate the next operating point
V_{base}/V_{max}	PI output, normalised	Tells the policy what the baseline already commands, this is what makes the control <i>residual</i>

$e_s = 5 \text{ rad/s}$, a moderately large tracking error; maps typical errors to ~ 1 , saturation errors to ~ 2

$\omega_s = 12 \text{ rad/s}$, just above the maximum reference (12 rad/s); keeps all speed features in $[-1, 1]$

All six features are clipped to $[-10, 10]$ before entering the network, no unbounded inputs

The observer is minimal by design: **6 scalars**, no history, no raw sensor data. The policy has exactly what it needs to compute the missing feedforward and nothing more.

Reward Function

The reward function is the only channel through which the policy learns what "good control" means:

$$r = -\alpha \frac{|e|}{e_s} - \lambda \left(\frac{\Delta V}{V_{max}} \right)^2 - \mu \left(\frac{\Delta V - \Delta V_{prev}}{V_{max}} \right)^2$$

It is computed at every policy step (every 10 ms) and has three terms:

First

Tracking Term

It penalises the **mean absolute speed error** averaged over the ten 1 ms sub-steps between policy updates.

Second

Effort term

It penalises **large residuals**, keeping the PI in charge and preventing the policy from issuing unnecessary corrections.

Third

Rate Term

It penalises how much the **correction changes** between consecutive updates, suppressing chattering and producing a smoother voltage profile.

We use the absolute value (L1) rather than the squared error (L2) because L1 weights small persistent errors more uniformly and is less sensitive to large transient spikes, on this problem it reduced the trajectory RMSE from 0.172 to 0.081 rad/s compared to L2.

The weights $\lambda = 0.01$ and $\mu = 0.05$ are roughly 100 times smaller than the tracking term ($\alpha = 1$), they shape the action without ever competing with the primary objective.

Training & Test set

Unlike supervised learning, there is no fixed dataset collected in advance. The policy trains *online*: at every step it interacts with the simulated motor, receives a **reward**, and updates its weights. The "dataset" is the distribution of speed references the environment samples during training.

Training set (12 references, fixed seed)

To evaluate the trained policy on in-distribution references we freeze a set of 12 profiles sampled from the same distribution used during training. Each profile is one of three types drawn at random:

- Constant, a fixed target speed held for the full 5 s episode;
- Quintic ramp, a smooth minimum-jerk transition from one speed to another, with random start time and duration;
- Multi-step, 2 to 4 piecewise-constant levels with random edges and amplitudes.

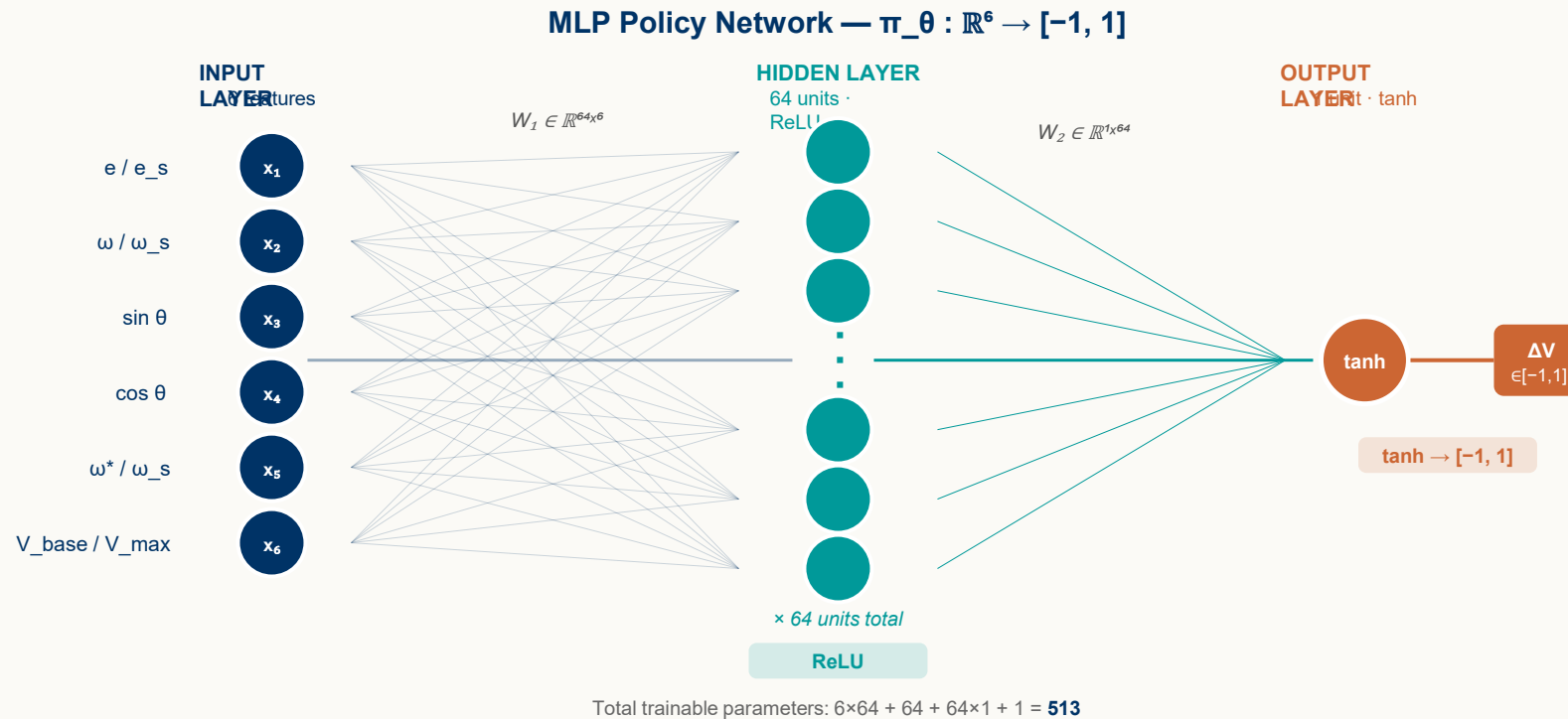
All references are drawn from $\omega \in [-12, 12] \text{ rad/s}$, matching the training range.

Test set (7 held-out references, never seen during training)

The test set is designed to probe three failure modes: constant references at 6, -5 and 12 rad/s cover different operating points including negative speeds; two quintic ramps (2→9 and 0→ -8 rad/s) and one step drop (8→3 rad/s) verify that generalisation holds across profile shapes and directions. The hardest probe is a constant at 14 rad/s , beyond the training range, where any performance is pure extrapolation and the strongest evidence against overfitting.

Neural Network Architecture

The correction ΔV the PI is missing depends on rod angle, operating speed, and reference profile in a **nonlinear** way and the input is a vector of six physical **scalars** with **no spatial structure** and **no temporal ordering**. An **MLP** is the natural choice: it learns an arbitrary smooth mapping from $\mathbb{R}^6$ to $\mathbb{R}^1$ with no inductive bias that would conflict with the problem structure.



One hidden layer and 64 units residual ΔV is a smooth, low-dimensional correction, this is sufficient to approximate it. ReLU activation, piecewise-linear, no vanishing gradient, standard for continuous-action SAC policies. $\tanh$ output, squashes the network output to $(-1, +1)$ regardless of weight magnitude, the residual limit $\Delta V \leq 3.6V$ is enforced by the architecture, not by clipping after the fact.

Saturation and Residual Authority: Design Choice

The residual authority $\Delta V = 0.3 \times V_{max} = 3.6 V$ is a deliberate design choice, not an optimised hyperparameter. The reasoning is physical: the two corrections the PI is missing are the gravity compensation and the steady-state feedforward. Together they sum to at most $4 - 5 V$ comfortably within the $\pm 3.6 V$ window. Setting δ too small would prevent the policy from correcting what the PI misses; setting it too large would let the policy override the baseline and risk destabilising the loop. 30% is the conservative choice that covers the physically meaningful corrections while keeping the PI in charge.

The saturation is managed at three levels:

Architecture

tanh bounds ΔV

The output neuron passes through *tanh*, so $a \in (-1, +1)$ always and $|\Delta V| = \delta |a| < 3.6 V$ regardless of weight values. The policy cannot saturate the residual channel even if it tries.

Control Law

Hard clip before the motor

$\text{clip}(V_{base} + \Delta V, -12, +12)$, PI and policy are summed first, then clipped. Neither component alone can push the motor beyond the hardware rail.

PI anti-windup

Prevents integrator blow-up

The integrator is clamped so $K_i \int e dt \leq \pm V_{max}$. Without this, a sustained error during a transient would wind the integrator to an arbitrarily large value, causing a slow oscillatory recovery once the error clears.

In our operating regime (references up to 12 rad/s), the total voltage touches the $\pm 12 V$ rail for only 0.1–0.4 % of the time, brief spikes at step changes. Saturation is not a limiting factor here. For higher speeds where rail contact becomes frequent, a saturation-aware observation mode is available: the state gains a seventh feature V_{prev}/V_{max} , the policy can sense the rail and avoid wasting effort against it

Machine Learning Validation: Train Vs Test set

This is a regression problem, the network outputs a continuous voltage correction, not a class label. We report four metrics on the tracking error $e = \omega - \omega^*$:

- MAE, mean absolute error: average magnitude of the tracking error
- RMSE, root mean squared error: primary control metric, penalises large deviations more
- R^2 , fraction of the reference variance explained by the achieved speed; defined only for time-varying references
- in-band %, fraction of time with $|e| < 0,5 \text{ rad/s}$; the honest analogue of "accuracy" for a continuous tracking task

Set	MAE	RMSE	R2	In-band %
Train (12 references)	0.069	0.292	0.955	99.7%
Test (7 held-out)	0.080	0.240	0.996	99.8%

Result. Test error is no worse than train on every metric, the policy has not memorised the training references, it has learned a general control strategy. R^2 improves on the test set because held-out references are smoother and easier to track than the random multi-step profiles used during training. The gap between train and test RMSE goes in the safe direction: no overfitting.

Ablation Study: Network Depth and Seed Robustness

To verify that the architecture choice is not arbitrary and the results are not seed-dependent, we trained 3 architectures $\times$ 3 random seeds = 9 independent models and evaluated each on the held-out test set. All models use the same reward, hyperparameters and training budget (300k steps).

Architecture	RMSE	R2	In-band %
MLP [64]	0.238 ± 0.001	0.996	99.8%
MLP [256,256]	0.256 ± 0.004	0.995	99.8%
MLP [256,256, 256]	0.247 ± 0.001	0.996	99.8%

Depth does not help. The smallest network achieves the lowest RMSE. Adding layers and parameters makes performance slightly worse, the residual is a smooth, low-dimensional function that a shallow network approximates just as well, with less risk of overfitting the training distribution.

The result is robust, not lucky. The per-seed standard deviation for [64] is ± 0.001 , negligible. The [256,256] network shows higher variance (± 0.004), meaning deeper models are more sensitive to initialisation. The multi-seed experiment confirms that the improvement over pure PI is a structural property of the residual architecture, not an artefact of a fortunate random seed.

Closed-Loop Performance: Benchmark

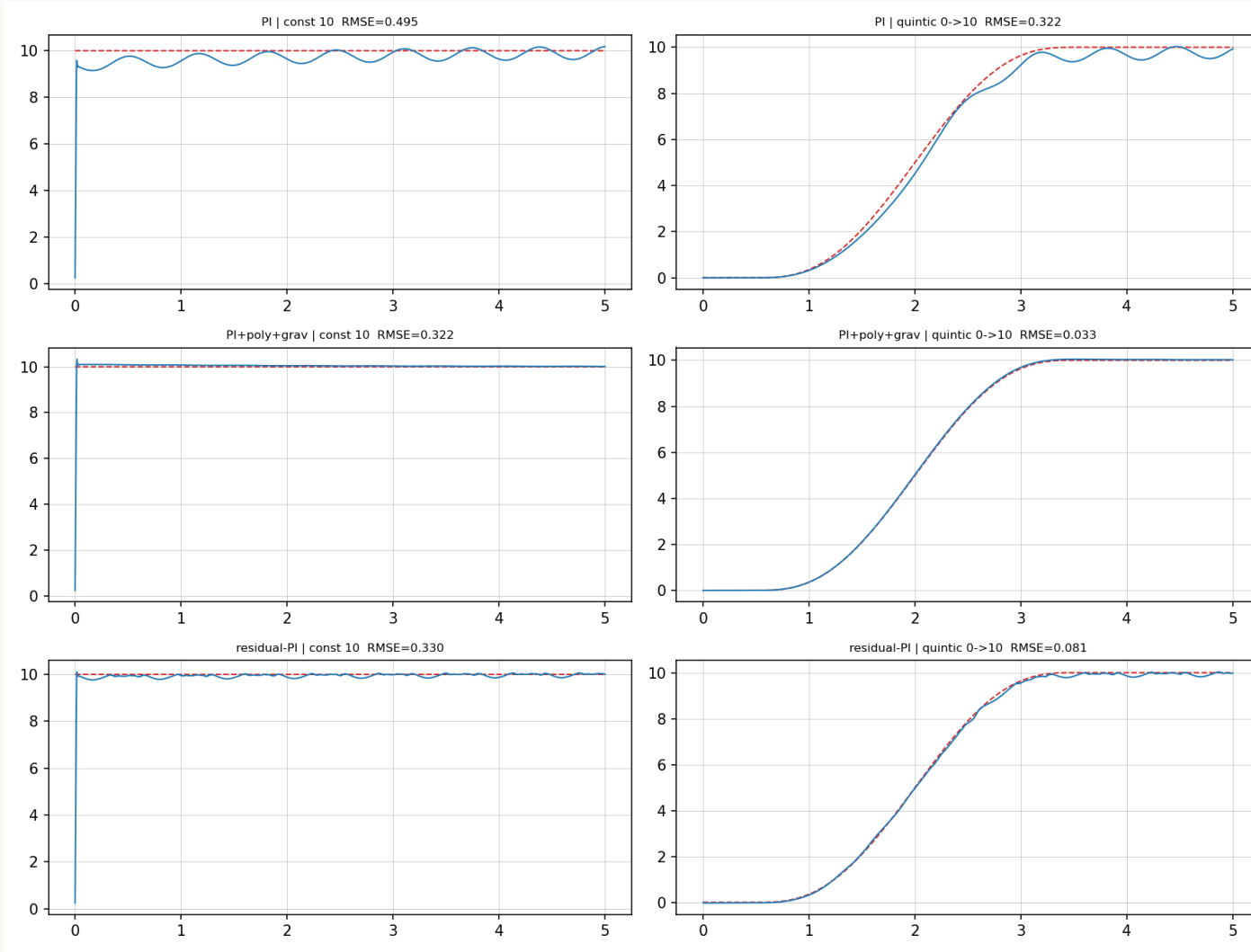
All three controllers run on the same simulated plant, same references, same noise model. Performance is measured on the held-out test set, 7 references never seen during training, to avoid any information leak between the RL training and the evaluation.

Controller	MAE	RMSE	R2	In-band %
PI (Baseline)	0.281	0.411	0.988	85.2%
PI + poly + grav	0.046	0.222	0.997	99.8%
RL residual	0.078	0.240	0.997	99.8%

RL residual reduces the pure PI tracking error by 75% (RMSE 0.411 $\rightarrow$ 0.240) and closes most of the gap to the strong classical baseline (0.240 vs 0.222), without any knowledge of the motor parameters or the gravity model. The in-band percentage jumps from 85% to 99.8%, matching the feedforward controller exactly on that metric.

The residual does not fully match PI + poly + grav on RMSE. This is expected: the classical controller uses the exact physical model; the policy approximates it from data alone. The remaining gap (0.018 rad/s) is the cost of being model-free.

Closed-Loop Performance: Controller Comparison



Each panel shows the achieved speed (blue) against the reference (dashed red) for one controller and one reference profile. RL Residual follows the reference closely on the quintic trajectory, where the PI alone shows a persistent steady-state offset, the learned ΔV compensates what the integrator cannot correct fast enough.

Closed-Loop Performance: Voltage Decomposition

Total motor voltage for each controller. Bottom: the RL residual split into V_{base} (PI baseline, blue) and ΔV (learned correction, orange). During transients ΔV is the dominant term, so the policy has implicitly learned the feedforward and gravity compensation that the PI baseline cannot provide.

